

Onderwijsgroep Professionele Opleidingen
Toegepaste Informatica

Master doc

L

Adam Bouchikhi (r1048050)
Arsalan Bornha (r1035349)
Larissa Mbonazo Lusakuono (r1029256)
Mohamed Amin Ahkim (r1038196)
Saartje Cordier (r1037117)
Tweede Bachelor Toegepaste Informatica

Integration project 2

Academiejaar 2025-2026

Eurovision Audit Systeem — Technisch Masterdocument

Odisee Hogeschool — Mega Data Challenge 2025-2026 Datum: 24 mei 2026

DEEL 1 — INFRASTRUCTUUR & VM-OVERZICHT

Google Cloud Project

- Project ID: `project-c15907ce-0bab-4a45-b70`
- Regio/Zone: `europa-west1-b`
- Totaal aantal VM's: 12
- VM-type: e2-medium (2 vCPU, 4 GB RAM — sommige hebben 8 GB)

VM-inventaris

VM	Intern IP	Extern IP	Rol
dc1-kafka	10.132.0.5	34.156.36.240	Kafka broker (centraal)
dc1-spark	10.132.0.6	34.140.166.209	Spark Master
dc1-cassandra	10.132.0.7	35.205.105.73	Cassandra + Hadoop
dc2-kafka	10.132.0.9	— (IAP)	Kafka producer node
dc2-spark	10.132.0.10	— (IAP)	Spark Worker
dc2-cassandra	10.132.0.8	— (IAP)	Cassandra node
dc3-kafka	10.132.0.14	— (IAP)	Kafka producer node
dc3-spark	10.132.0.16	— (IAP)	Spark Worker
dc3-cassandra	10.132.0.12	— (IAP)	Cassandra node
dc4-kafka	10.132.0.15	— (IAP)	Kafka producer node
dc4-spark	10.132.0.17	— (IAP)	Spark Worker

dc4-cassandra	10.132.0.13	— (IAP)	Cassandra node
---------------	-------------	---------	----------------

Netwerconnectiviteit: dc1-* heeft een extern IP en directe verbinding. dc2/dc3/dc4 communiceren via IAP-tunnel (beheer) en intern netwerk 10.132.0.x (dataverkeer).

DEEL 2 — DOCKER CONTAINERS PER VM

dc1-cassandra (meest beladen node)

Containernaam	Image	Poorten	Status
eurovision-cassandra	cassandra:4.1	9042 (CQL)	Up
eurovision-datanode	hadoop-datanode	9864	Up (healthy)
eurovision-namenode	hadoop-namenode	9000, 9870	Up (healthy)

dc1-kafka

Containernaam	Image	Poorten	Status
eurovision-kafka	kafka	9092	Up
eurovision-zookeeper	zookeeper	2181	Up

dc1-spark

Containernaam	Image	Poorten	Status
larissa_mbonazolusakueno-spark-master-1	spark	7077/7078, 8080	Up
spark-worker	spark	8081	Up

DEEL 3 — POORTEN & SERVICES

Poort	Service	VM	Protocol
-------	---------	----	----------

9092	Kafka broker	dc1-kafka (10.132.0.5)	TCP
2181	Zookeeper	dc1-kafka	TCP
9042	Cassandra CQL	dc1-cassandra (10.132.0.7)	TCP
7077	Spark Master (intern)	dc1-spark	TCP
7078	Spark Master URL	dc1-spark	TCP
8080	Spark Master UI	dc1-spark	HTTP
8081	Spark Worker UI	dc2/3/4-spark	HTTP
9000	Hadoop HDFS NameNode	dc1-cassandra	TCP
9870	Hadoop NameNode UI	dc1-cassandra	HTTP
9864	Hadoop DataNode	dc1-cassandra	TCP

Spark Master URL: `spark://10.132.0.6:7078` **Kafka Bootstrap Server:** `10.132.0.5:9092` **Cassandra Host:** `localhost` (op dc1-cassandra) of `10.132.0.7` (vanuit andere VM's)

DEEL 4 — KAFKA CONFIGURATIE

Topic

- Naam: `eurovision-votes`
- Partities: **12** (uitgebreid van 1 → 12, één per VM)
- Replication Factor: 1001 (zoals zichtbaar in de topic describe output)
- Leader: 1001
- ISR: 1001

Producer configuratie (producer.py)

Parameter	Waarde	Reden
-----------	--------	-------

batch_size	65536 bytes	Grotere Kafka-batch buffer
linger_ms	5 ms	Wacht 5 ms om berichten te groeperen
compression_type	lz4	Snelle compressie, ~60% minder netwerk-overhead
acks	1	Alleen leader bevestigt (sneller dan acks=all)
NUM_WORKERS	8 threads	Parallel versturen per VM
TOTAL_VOTES	100.000	Per run per VM (load test)
BATCH_SIZE	500	Berichten per flush

Data schema per Kafka-event (JSON)

json

```
{
  "country": "Belgium",
  "timestamp": 1748110408.123
}
```

DEEL 5 — CASSANDRA SCHEMA

Keyspace

sql

```
CREATE KEYSPACE eurovision ...;
```

Tabel: **eurovision.votes**

sql

```
CREATE TABLE eurovision.votes (
  country TEXT,
  timestamp TIMESTAMP,
  vote_id UUID,
  voter_ip TEXT,
  PRIMARY KEY (country, timestamp, vote_id)
);
```

Uitleg van de primary key:

- **country** = partition key — alle stemmen per land zitten fysiek samen op dezelfde node. Dit maakt COUNT per land via de partition key zeer snel en betrouwbaar.
- **timestamp** + **vote_id** = clustering columns — zorgen voor unieke rijen en vermijden duplicaten (UUID is altijd uniek).

Waarom geen COUNTER type: De tabel slaat individuele stemmen op (1 rij = 1 stem), niet geaggregeerde tellingen. Dit laat toe de ruwe data te herberekenen voor de consistency check.

Aggregatie-methode: Cassandra ondersteunt geen **SUM()** op grote datasets zonder partition key zonder timeout. De oplossing is **COUNT(*) WHERE country = 'X'** per land afzonderlijk, en de resultaten optellen in Python.

DEEL 6 — SPARK CLUSTER

Topologie

Node	VM	IP	Rol	Cores	RAM
Master	dc1-spark	10.132.0.6	Coördinator	—	—
Worker 1	dc2-spark	10.132.0.10	Verwerker	2	6.8 GiB
Worker 2	dc3-spark	10.132.0.16	Verwerker	2	6.8 GiB
Worker 3	dc4-spark	10.132.0.17	Verwerker	2	6.8 GiB

Totale cluster capaciteit: 6 cores, ~20 GiB RAM

Opmerking: Tijdens de tests was er geen actieve Spark Structured Streaming job die van Kafka las en naar Cassandra schreef. De data-integriteitstest is daarom uitgevoerd via directe Cassandra-insert (Manier B). Dit is een PoC-beperking, geen architectuurfout.

DEEL 7 — HADOOP CONFIGURATIE

Hadoop draait samen met Cassandra op dc1-cassandra. Dit is de audit trail component.

Component	Container	Poort
-----------	-----------	-------

NameNode	eurovision-namenode	9000 (HDFS), 9870 (UI)
DataNode	eurovision-datanode	9864

Rol in het systeem: Hadoop is bedoeld als onafhankelijke opslag van ruwe stem-events voor de audit trail. In de PoC wordt de consistency check gesimuleerd door dezelfde Cassandra-data twee keer op te vragen via twee verschillende query-methodes.

DEEL 8 — Technische ontwerpkeuzes & motivatie

Waarom Apache Kafka? Tijdens de stemperiode kunnen plots grote pieken ontstaan — miljoenen kijkers stemmen tegelijk in de laatste minuten. Een directe verbinding van producer naar Spark zou bij zo'n piek alles laten crashen. Kafka fungeert als buffer die pieken absorbeert en de data aan een stabiel tempo doorgeeft aan Spark, ongeacht de instroom. Kafka is ook gedistribueerd van nature: met 12 partities kunnen 12 producers parallel schrijven zonder elkaar te blokkeren.

Waarom 12 partities? Eén partitie per VM. Dit zorgt dat elke producer naar zijn eigen partitie schrijft en er geen lock-contentie is op de Kafka broker. In productie zou je één Kafka broker per datacenter hebben — de huidige setup met één centrale broker is een bekende PoC-beperking.

Waarom LZ4 compressie in de producer? LZ4 reduceert de netwerkoverhead met approximately 60% bij JSON-data. Omdat de producers via het interne netwerk communiceren is bandbreedte beperkt, en LZ4 is de snelste compressie-algoritme voor real-time gebruik — nauwelijks CPU-kost maar significant minder bytes over het netwerk.

Waarom Apache Cassandra en niet MySQL of PostgreSQL? Cassandra is ontworpen voor hoge schrijfsnelheid op grote schaal. Relationale databases zoals MySQL gebruiken ACID-transacties en rij-locks die bij gelijktijdige massale writes een bottleneck worden. Cassandra schrijft altijd naar het einde van een append-only log, wat extreem snel is. Bovendien is de partition key (**country**) de ideale structuur voor onze query: we willen altijd per land tellen, nooit een full table scan.

Waarom elke stem als aparte rij en niet als teller? Een Cassandra COUNTER-tabel zou sneller zijn voor het ophalen van totalen, maar verliest de mogelijkheid om individuele stemmen te verifiëren. Door elke stem als aparte rij op te slaan met een unieke **vote_id** (UUID) is het mogelijk om duplicaten te detecteren, per tijdstip te analyseren, en de volledige stemgeschiedenis te herberekenen. Dit is essentieel voor een auditsysteem.

Waarom Google Cloud en niet Oracle Cloud? Oracle Cloud in combinatie met KVM-virtualisatie werd initieel onderzocht maar kon niet correct worden gevalideerd met de beschikbare studentenaccounts. Google Cloud bood voldoende quota voor 12 e2-medium VM's in één project, met IAP-tunneling voor veilige toegang tot VM's zonder extern IP.

Waarom Hadoop naast Cassandra? Cassandra is geoptimaliseerd voor snelle reads en writes op actuele data maar is minder geschikt voor lange-termijn archivering en heranalyse van grote historische datasets. Hadoop/HDFS bewaart de ruwe events ongewijzigd, waardoor je maanden later nog een exacte herberekening kunt doen. Dit is de scheiding tussen operationele data (Cassandra) en historische audit data (Hadoop)

DEEL 9 — Dataflow

Een stem doorloopt de volgende stappen:

Stap 1 — Generatie (producer.py op elke VM) Een Python-script simuleert een inkomende SMS-stem. Het genereert een JSON-object met `country` en `timestamp` en stuurt dit via het interne netwerk (10.132.0.x) naar de Kafka broker op dc1-kafka:9092. Elke van de 12 VM's heeft een eigen producer met 8 threads en een batchgrootte van 500 berichten.

Stap 2 — Buffering (Apache Kafka op dc1-kafka) Kafka ontvangt de stem en plaatst hem in het topic `eurovision-votes`. Met 12 partities schrijft elke producer naar een andere partitie, waardoor er geen wachtrij ontstaat. Kafka fungeert als buffer: zelfs bij een piek van miljoenen stemmen per seconde gaat er niets verloren, want Kafka houdt alles bij totdat Spark klaar is om te lezen.

Stap 3 — Verwerking (Apache Spark) Spark Structured Streaming leest de events uit Kafka, telt de stemmen per land en berekent de ranking. De Spark Master op dc1-spark coördineert; de Workers op dc2/dc3/dc4-spark voeren de verwerking parallel uit. In de PoC was deze koppeling nog niet actief — de data-integriteitstest is daarom via directe Cassandra-insert uitgevoerd.

Stap 4 — Opslag (Apache Cassandra op dc1-cassandra) Elke stem wordt als aparte rij opgeslagen in `eurovision.votes`. De partition key is `country`, wat betekent dat alle stemmen per land fysiek bij elkaar staan en snel optelbaar zijn via `COUNT(*) WHERE country = X`.

Stap 5 — Audit (Hadoop op dc1-cassandra) Hadoop draait op dezelfde VM als Cassandra en dient als onafhankelijke audit trail. In de PoC simuleert de consistency check dit door dezelfde data via een alternatieve query-methode te hertellen. In productie zou Hadoop de ruwe events onafhankelijk bewaren voor herberekening.

Stap 6 — Verificatie (demo-scripts) `demo_integriteit.py` bewijst dat geen enkele stem verloren gaat (NFR-03). `demo_consistency.py` bewijst dat het scorebord overeenkomt met de onafhankelijke herberekening (NFR-04).

DEEL 10 — SCRIPTS OVERZICHT

1. producer.py — Load Testing

Locatie: `~/producer.py` op alle 12 VM's **Functie:** Stuurt stemmen naar Kafka topic `eurovision-votes`

Belangrijke variabelen:

Variabele	Waarde	Beschrijving
BOOTSTRAP	10.132.0.5:9092	Kafka broker intern IP
TOPIC	eurovision-votes	Kafka topic naam
NUM_WORKERS	8	Threads per VM
TOTAL_VOTES	100.000	Stemmen per run
BATCH_SIZE	500	Berichten per batch
MEASURE_WINDOW	5 sec	Meetvenster throughput

Output: Geeft per meetvenster de actuele en gemiddelde throughput, plus percentage van het 200K/sec doel.

2. demo_integriteit.py — Data Integriteit (NFR-03)

Locatie: `/home/larissa_mbonazolusakueno/demo_integriteit.py` op dc1-cassandra **Functie:** Bewijst dat Cassandra geen data verliest

Werking in 5 stappen:

- Stap 0: TRUNCATE votes (schone start)
- Stap 1: Tel VOOR (verwacht: 0)
- Stap 2: Voeg exact 1.000.000 stemmen in (batch_size=1000)
- Stap 3: Tel NA per land via partition key
- Stap 4: Verificatie verschil == 1.000.000

Testresultaat:

Metric	Waarde
Stemmen VOOR	0

Stemmen NA	1.000.000
Dataverlies	0
Invoerduur	2.470 seconden (~41 min)
Uitvoeringstijd	41 min 20 sec
Gestart	2026-05-24 19:06:08
Gestopt	2026-05-24 19:47:28

Verdeling per land:

Land	Stemmen
Belgium	99.873
Netherlands	99.837
France	99.817
Germany	100.106
Italy	99.845
Spain	100.122
Sweden	99.880
Norway	100.189
Ukraine	100.344
Portugal	99.987

TOTAAL	1.000.000
---------------	------------------

3. demo_consistency.py — Consistency Check (NFR-04)

Locatie: `/home/larissa_mbonazolusakueno/demo_consistency.py` op dc1-cassandra **Functie:** Vergelijkt scorebord (Cassandra live scores) met onafhankelijke herberekening (simuleert Hadoop audit)

Werking:

- Stap 1: Lees scores per land via `COUNT(*) WHERE country=%s`
- Stap 2: Herbereken via `COUNT(*) WHERE country=%s ALLOW FILTERING`
- Stap 3: Vergelijk per land, tel afwijkingen
- Stap 4: Auditrapport

Testresultaat:

Metric	Waarde
Gevalideerde landen	10/10
Afwijkingen	0
Totaal scorebord	1.000.000
Totaal herberekend	1.000.000
Uitvoeringstijd	9 seconden
Gestart	2026-05-24 20:43:14
Gestopt	2026-05-24 20:43:24

4. demo.sh — Orchestratie Script

Locatie: `~/demo.sh` op dc1-kafka (of Cloud Shell) **Functie:** Start alle 12 producers tegelijk, controleert Kafka en Spark, meet throughput en bottleneck

Stappen in het script:

1. Kafka topic check (12 partities bevestigen)
2. Spark master log check (workers geregistreerd)
3. Alle 12 producers starten via **nohup** (achtergrond)
4. 15 seconden wachten
5. Throughput uitlezen per VM uit **producer_output.txt**
6. CPU/RAM bottleneck meten op de 4 Kafka-nodes

DEEL 11 — LOAD TEST RESULTATEN

Throughput per VM (individueel gemeten)

VM	Events/sec	% van 200K doel
dc1-kafka	1.281.318	612% → ~1.2M/sec
dc1-spark	2.400.861	887% → ~1.8M/sec
dc1-cassandra	2.637.512	905% → ~1.8M/sec
dc2-kafka	2.526.050	997% → ~2.0M/sec
dc2-spark	1.971.638	1017% → ~2.0M/sec
dc2-cassandra	2.500.584	1428% → ~2.9M/sec
dc3-kafka	2.359.625	1199% → ~2.4M/sec
dc3-spark	2.238.622	1019% → ~2.0M/sec
dc3-cassandra	2.362.230	1105% → ~2.2M/sec
dc4-kafka	2.005.002	899% → ~1.8M/sec
dc4-spark	2.363.294	1261% → ~2.5M/sec
dc4-cassandra	2.009.132	1128% → ~2.3M/sec

TOTAAL	~26.655.868	13.300%
---------------	--------------------	----------------

Conclusie: 133× het doel van 200K/sec behaald.

Bottleneck analyse

Component	Bevinding
CPU	Geen probleem — max 55% op dc4-cassandra, rest onder 10%
RAM	Geen probleem — Cassandra gebruikt ~2 GB van 4 GB
Netwerk data	Geen bottleneck — ~370 MB/sec intern verkeer
Bottleneck	Kafka broker: 1 broker voor 12 producers tegelijk

Netwerkttype verschil:

- dc1-kafka: directe verbinding (extern IP) → ~48 KB/sec beheerverkeer
- dc2/3/4-kafka: IAP-tunnel → ~50–60 KB/sec beheerverkeer
- Datavlak (10.132.0.x intern): geen tunneling, volledige bandbreedte beschikbaar

DEEL 12 — Testmethodologie

Wat getest is en hoe

De load test (throughput) is uitgevoerd door op alle 12 VM's tegelijk het `producer.py` script te starten via `demo.sh`. Elke VM stuurde 100.000 stemmen via 8 threads naar de centrale Kafka broker. De throughput is per VM afzonderlijk gemeten via een voortschrijdend gemiddelde over een meetvenster van 5 seconden. Het totaal van ~26.6M events/sec is de som van alle 12 individuele metingen — niet één gecombineerde meting. Dit onderscheid is belangrijk: in productie zou de gecombineerde throughput lager liggen door de Kafka broker als centrale bottleneck.

De data-integriteitstest is uitgevoerd via directe Cassandra-insert, niet via de volledige Kafka→Spark→Cassandra pipeline. Dit is een bewuste keuze omdat de Spark Streaming job nog niet operationeel was. De test bewijst dataverlies in de opslaglaag (Cassandra), niet in de volledige pipeline.

De consistency check is uitgevoerd op dezelfde dataset als de integriteitstest (1.000.000 stemmen, schone tabel). De "scorebord"-waarden en de "herberekende" waarden komen uit dezelfde Cassandra-tabel via twee verschillende query-methodes. In productie zouden deze uit twee onafhankelijke systemen komen (Cassandra en Hadoop).

Aannames en randvoorwaarden

Alle tests zijn uitgevoerd op e2-medium VM's (2 vCPU, 4 GB RAM), wat significant kleiner is dan productieinfrastructuur. De stemverdeling per land is random uniform verdeeld over 10 landen — in werkelijkheid zou de verdeling ongelijk zijn. De VM crashte éénmaal tijdens de integriteitstest door geheugengebrek, wat bevestigt dat de VM-grootte een beperkende factor is. Na herstart was alle data bewaard, wat de duurzaamheid van Cassandra's opslag aantoonst.

Requirements traceerbaarheid

Requirement	Test	Resultaat
F-R01 — stemdata opslaan	demo_integriteit.py	1.000.000 rijen opgeslagen
F-R02 — inkomende stemmen verwerken	producer.py + demo.sh	26.6M events/sec
F-R03 — correcte ranking berekenen	demo_consistency.py	10/10 landen exact
F-R05 — controles uitvoeren	demo_consistency.py	NFR-04 voldaan
NF-R01 — schaalbaar	Load test 12 VM's	133× het doel
NF-R03 — betrouwbare resultaten	demo_integriteit.py	0 dataverlies
NF-R04 — robuust bij hoge belasting	Load test	CPU max 55%, RAM stabiel

Hiermee is het masterdocument compleet. De drie aanvullingen los je de drie lacunes op die ik eerder had geïdentificeerd: de dataflow is nu tekstueel gedocumenteerd, de ontwerpkeuzes zijn gemotiveerd vanuit jullie eigen projectrapport, en de testmethodologie is formeel beschreven inclusief de eerlijke kanttekening over wat de metingen wel en niet bewijzen.

DEEL 13 — NFR VERIFICATIE

NFR	Beschrijving	Status	Bewijs
-----	--------------	--------	--------

NFR-01	200.000 events/sec piekverkeer	Ok	26.6M/sec gemeten — 133× doel
NFR-03	Geen dataverlies	Ok	1.000.000 in, 1.000.000 uit — 0 verlies
NFR-04	Scorebord == Herberekening	Ok	10/10 landen exact match, 9 sec uitvoering

DEEL 14 — BEKENDE BEPERKINGEN (PoC)

1. Spark Streaming job niet actief: Er was geen actieve Spark Structured Streaming job die van Kafka las en naar Cassandra schreef. De data-integriteitstest is daarom uitgevoerd via directe Cassandra-insert. In productie zou de volledige pipeline (Kafka → Spark → Cassandra) actief zijn.

2. Hadoop simulatie: De consistency check simuleert de Hadoop-herberekening door dezelfde Cassandra-tabel twee keer te bevragen via een andere query-methode (**ALLOW FILTERING**). In productie zou Hadoop de ruwe events onafhankelijk opslaan en hertellen.

3. VM-formaat: e2-medium (2 vCPU, 4 GB RAM) is klein voor productie. De VM crashte éénmaal door geheugengebrek na 1M inserts. Na herstart was alle data bewaard (Cassandra persistentie bewezen).

4. Enkelvoudige Kafka broker: In productie zou je per datacenter een Kafka broker hebben, niet één centrale broker voor alle 12 producers. Dit is de geïdentificeerde architectuurverbetering.

5. Geen extern IP op dc2/3/4: Deze VM's communiceren via IAP-tunnel voor beheer. Het datavlak gaat intern en is niet gelimiteerd door de tunnel.